Juno Intents: Threshold-Operated Bridging for Shielded Assets

Abdullah R, Romano (@RNR_0)
April 2026

Abstract

We present Juno Intents, a bridge architecture for extending shielded Junocash liquidity to an EVM-compatible chain without reducing custody to a single bridge signer. The architecture is EVM-generic; the current deployment targets Base, a Layer 2 network chosen for its lower transaction costs and higher throughput. The system combines threshold-controlled custody on Junocash, operator-signed checkpoints on the EVM side, SP1-backed proof verification relative to those checkpoints, and published viewing material for external monitoring.

The EVM chain does not verify Junocash consensus directly. The bridge therefore depends on an operator quorum not signing an invalid checkpoint. Funds are released only after verification of a valid proof against an accepted threshold-signed checkpoint together with the corresponding settlement checks.

We describe a bridge model in which custody, attestation, proving, and settlement are separated into explicit stages with defined trust boundaries, publicly inspectable data, and governance mechanisms for pause, rotation, and recovery. We analyze the trust model, failure modes, and operating envelope, and we compare the design to conventional single-signer and light-client bridge architectures.

1 Introduction

1.1 Background

Shielded-asset bridging differs materially from bridging in transparent systems. In a conventional transparent bridge, deposits are visible on-chain, can be indexed directly, and can often be verified against an external chain with relatively straightforward logic [8]. In a mandatory-privacy Orchard-based system such as Junocash [2], none of these properties hold. The destination chain does not see a public deposit record, does not natively understand Junocash state transitions, and cannot cheaply determine whether a particular shielded note should authorize minting elsewhere.

Junocash enforces that all transactions occur within the Orchard shielded pool [4], with no transparent address type and no transparent pool. Every note, including coinbase outputs to miners, is shielded from genesis. The bridge therefore operates over a chain in which the entire transaction volume is private, which both strengthens the anonymity set available to bridge users and makes external auditability a deliberate design requirement rather than an automatic property of chain transparency.

The design problem is accordingly broader than asset transfer across two chains. It is the movement of value out of a shielded state machine while preserving privacy, constraining custody risk, and still providing enough evidence for external audit. If those three goals are not addressed together, the result is typically either an opaque custodial bridge or a system that overstates trust minimization while depending on trusted operators.

Juno Intents addresses this setting directly. Base accepts typed checkpoints of Junocash state signed by a threshold of registered operators. Zero-knowledge proofs are then verified relative

to those accepted checkpoints. Custody of the Junocash-side bridge wallet is itself threshold-controlled through distributed key generation, and viewing material is published so that external observers can monitor bridge activity without receiving spend authority.

1.2 Contributions

We make the following contributions:

Threshold Custody: We employ distributed key generation so that no single operator holds spend authority over the Junocash-side bridge wallet. The wallet is born under threshold control and can be renewed through controlled rotation ceremonies.

Proof-Backed Settlement: We use SP1 zero-knowledge proofs [7] to bind deposit and withdrawal claims to signed Orchard roots, replacing unsupported relayer testimony with proof-carrying claims that Base can verify against accepted checkpoints.

Public Auditability: We publish a Unified Full Viewing Key derived deterministically from the threshold group key, enabling independent observation of bridge wallet activity without exposing spend authority.

Explicit Trust Boundary: We state directly that the bridge is threshold-operated rather than trustless. Base acts on signed Junocash state commitments rather than direct consensus verification. We analyze the implications of that boundary and describe the governance mechanisms that surround it.

Table 1 summarizes the guarantees, assumptions, and external dependencies of the design.

Table 1: Summary of guarantees, assumptions, and external dependencies.

Lens	**Description**
Guarantees	No single operator can spend from the Junocash-side bridge wallet alone. Base accepts deposit and withdrawal claims only when they are tied to a signed Junocash checkpoint and a valid proof relative to that checkpoint. Published viewing material makes note flows independently auditable.
Assumptions	Base does not verify Junocash consensus directly. A malicious checkpoint quorum can sign a false checkpoint. Availability depends on at least t operators, durable key-share storage, and proof infrastructure remaining live.
Dependencies	Threshold custody ceremonies, operator key management, and proof infrastructure remain active operational components.

The remainder of this paper is organized as follows. Section 2 decomposes the design objectives. Section 3 introduces the system model and notation. Section 4 describes the architecture. Sections 5 and 6 specify the deposit and withdrawal protocols. Section 7 analyzes the trust model and verification boundary. Section 8 describes the proof architecture. Section 9 covers distributed custody and auditability. Section 10 addresses operator economics and governance. Section 11 discusses ceremony evidence and recoverability. Section 12 analyzes reliability and failure modes. Section 13 positions the design relative to alternative bridge architectures. Section 14 concludes.

2 Problem Setting and Design Objectives

Juno Intents addresses the problem of moving Juno between the shielded Junocash environment and an EVM chain without collapsing either privacy expectations or operational control into a single opaque actor. We decompose this problem into four requirements that are often conflated in bridge design.

First, custody on the Junocash side should not reduce to one hot wallet or one trusted signer. If a single operator can spend from the bridge wallet alone, the bridge becomes custodial in the narrowest sense, and every other mechanism in the design sits downstream of a single-point custody failure.

Second, Base-side minting and withdrawal finalization should depend on explicit Junocash state commitments rather than ad hoc relayer testimony. A relayer may carry data from one side to the other, but it should not be the authoritative source of whether a deposit or withdrawal is valid.

Third, bridge behavior should be externally auditable. Because Junocash enforces mandatory privacy with no transparent pool [2], outsiders cannot reconstruct bridge activity from public chain data the way they might with a transparent bridge. A bridge that moves shielded value must therefore create public audit surfaces deliberately. Published viewing material, signed checkpoint packages, and proof-carrying settlement inputs serve this purpose.

Fourth, the system must admit governance and recovery. If operators equivocate, if custody material is suspected compromised, or if proving infrastructure degrades, the bridge should support pause, signer rotation, verifier updates, and renewed custody ceremonies.

These requirements explain why we design Juno Intents as threshold-operated rather than trustless, and as operationally renewable rather than static.

3 System Model

3.1 Notation and Primitives

We collect the notation used throughout this paper. Table 2 names the exact objects the bridge reasons about.

Table 2: Principal notation.

Symbol	Meaning
n	Number of registered operators in the active keyset
t	Threshold for checkpoint quorum and threshold spend authorization
$O = \{1, 2, \ldots, n\}$	Operator identifier set
$S \subseteq O$	Subset of operators participating in one signing event
C_h	Checkpoint tuple for Junocash height h
ak	Group spend validating key for threshold custody
UFVK	Published unified full viewing key material

The checkpoint tuple for Junocash height h is

$$C_h = \big(h,\ \texttt{blockHash}_h,\ \texttt{finalOrchardRoot}_h,\ \texttt{baseChainId},\ \texttt{bridgeContract}\big). \tag{1}$$

The threshold checkpoint condition is

$$|S| \geq t \quad \text{with all signer identities in } S \text{ distinct.} \tag{2}$$

The checkpoint digest is an EIP-712 [6] typed hash over those fields:

$$\mathrm{Digest}(C_h) = \mathrm{Keccak256}\big(\texttt{0x1901} \parallel \mathrm{domainSeparator} \parallel \mathrm{structHash}(C_h)\big). \tag{3}$$

3.2 Layered System Model

Juno Intents spans three layers. On Junocash, a threshold-controlled bridge wallet receives shielded deposits and produces threshold-authorized payout transactions. Between the chains,

operators and support services maintain checkpoint, witness, and proof infrastructure that converts Junocash events into artifacts Base can evaluate. On Base, a bridge contract, operator registry, wrapped asset contract, and fee distribution logic accept signed checkpoints, verify proofs, mint or burn `wJUNO`, and track withdrawal state.

Table 3 identifies the principal actors.

Table 3: Principal actors in the Juno Intents system.

Actor	**Role**
User	Deposits on Junocash or requests withdrawal on Base
Operator quorum	Signs checkpoints, participates in threshold custody and payout authorization
Relayer / submitter	Submits mint and finalization transactions on Base
Proof services	Build guest inputs, request proofs, return seals tied to specific journals
Base contracts	Verify checkpoints and proofs, escrow or burn `wJUNO`, distribute fees

Two settlement lanes define the protocol. The Junocash-to-Base lane turns a shielded Juno deposit into a proof-backed mint of `wJUNO` on Base. The Base-to-Junocash lane escrows `wJUNO`, coordinates a threshold-authorized payout on Junocash, and settles the record on Base after proof-backed confirmation. The important architectural property is stage separation: custody, checkpoint attestation, proof generation, and contract settlement are different stages with different failure modes and different audit surfaces.

4 Architecture Overview

Figure 1 illustrates the system architecture. The Junocash-to-Base lane begins with shielded note discovery and ends with minting on Base. The Base-to-Junocash lane begins with escrow on Base and ends with a proved and finalized withdrawal after Junocash payout confirmation. The asymmetry between these two directions is intentional: deposit minting depends on recognizing a bridge-eligible note inside shielded state, whereas withdrawal finalization depends on a prior escrow record and later confirmation that the Junocash payout has already occurred.

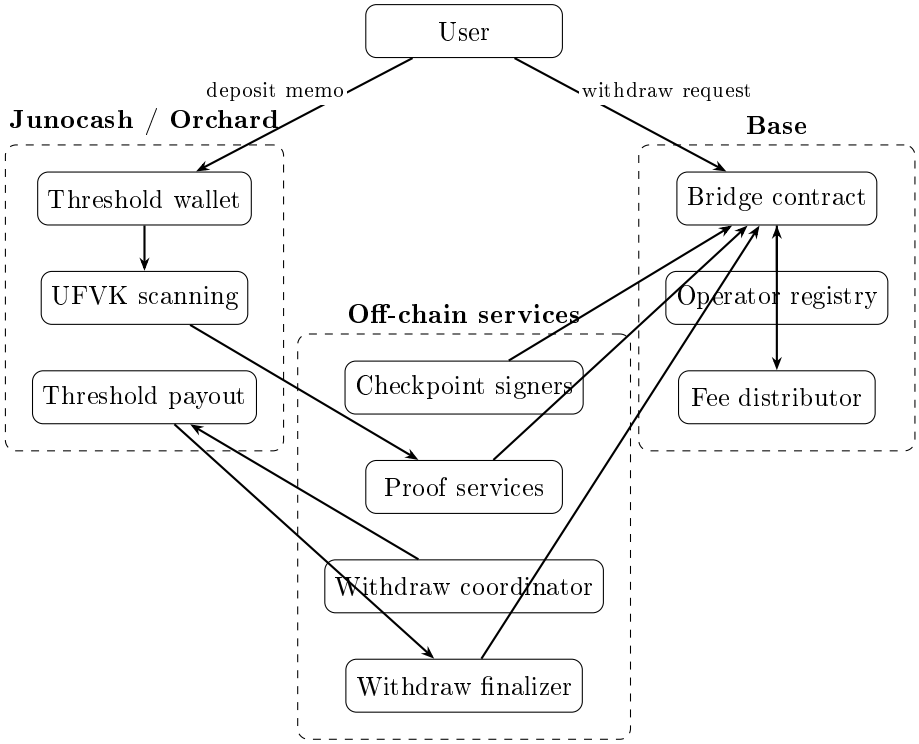

Figure 1: High-level architecture of Juno Intents. Arrows indicate the principal data and control flows. Dashed boundaries group components by execution domain.

Checkpoint signing and proof verification serve distinct roles in this architecture. The checkpoint supplies the state commitment that Base is willing to trust. The proof shows that a specific deposit or withdrawal event is consistent with that committed state. Off-chain services remain necessary because Junocash events must be observed, packaged, and proved before Base can evaluate them.

5 Junocash-to-Base Deposit Protocol

A Junocash-to-Base transfer begins when the bridge wallet receives a shielded deposit whose memo routes it to a specific Base deployment. A shielded note is not automatically a bridge deposit; it becomes bridge-eligible only if it can be interpreted under the bridge's viewing surface and memo rules. We do not accept a deposit because a relayer asserts that one exists. We accept a deposit because the note can be recovered under the bridge viewing surface, tied to a particular Orchard root, and interpreted under a bridge-specific memo format.

5.1 Memo Format and Domain Separation

The routing payload is `DepositMemoV1`. Its function is domain separation: it prevents a note intended for one destination or one bridge deployment from being silently reused in another context. The memo binds the deposit to a Base chain and a bridge contract before any mint semantics are derived. Memo parsing checks a fixed-width format, CRC32 checksum discipline, zero padding, chain binding, and bridge binding. The full encoding is specified in Appendix A.

5.2 Canonical Deposit Identifier

The deposit identifier is canonical and derived from the note commitment and Orchard leaf index rather than from relayer-controlled metadata:

$$\texttt{depositId} = \text{Keccak256}(\texttt{"deposit"} \parallel \texttt{cmx} \parallel \texttt{leafIndex}_{\text{BE32}}).^{1} \tag{4}$$

This construction ensures that replay resistance does not depend on relayer-chosen metadata.

5.3 Deposit Proving Path

Once a witness is assembled, the relayer obtains a signed checkpoint package for the Orchard root that the proof will target, constructs the guest input, and requests a proof over the exact journal that Base will later verify. At a high level, the deposit proving path establishes three facts: the note is included under the signed Orchard root, it decrypts under the bridge incoming viewing key, and its memo yields a valid (`depositId`, `recipient`, `amount`) record for the target Base deployment.

5.4 Base Settlement

The Base contract executes `mintBatch`. For each proved item, it verifies the checkpoint quorum, verifies the proof, checks journal-domain agreement with the checkpoint, and applies bridge policy including replay protection and minimum amount rules. Accepted items mint net `wJUNO` to the recipient, route protocol fees to operator accounting, and route a relayer tip to the transaction submitter. The protocol prefers partial forward progress under a correct checkpoint to whole-batch reversion when a mixed batch contains replayed or malformed items.

Table 4 summarizes the deposit stages.

Table 4: Deposit settlement stages.

Stage	Action	Main check
Scan	Deposit note is detected	Note recoverable under UFVK
Memo parse	`DepositMemoV1` is decoded	Domain binding, checksum, padding
Proof	Guest constructs a deposit journal	Inclusion, decryption, memo semantics
Settlement	`mintBatch` executes on Base	Checkpoint quorum, proof, replay, minimums

Base does not decide whether a Junocash deposit exists by reading Junocash consensus directly. It decides whether a proved deposit is admissible relative to a signed checkpoint that the bridge has already accepted.

6 Base-to-Junocash Withdrawal Protocol

The reverse direction begins on Base. `requestWithdraw` escrows `wJUNO`, records the raw Junocash recipient bytes, snapshots fee state, sets an expiry window, and derives a canonical withdrawal identifier:

$$\begin{aligned}\texttt{withdrawalId} = \text{Keccak256}\big(&\texttt{"WJUNO_WITHDRAW_V1"} \parallel \texttt{chainId} \parallel \texttt{bridge} \\ &\parallel \texttt{nonce} \parallel \texttt{requester} \parallel \texttt{amount} \parallel \text{Keccak256}(\texttt{recipientUA})\big).^{2}\end{aligned} \tag{5}$$

This stage fixes the economic and routing state of the withdrawal before any Junocash payout is attempted.

[1] The deposit guest program also accepts an ASCII-hex-wrapped compact compatibility form to support standard wallet memo encoding.

[2] The on-chain implementation uses ABI encoding, not raw byte concatenation.

6.1 Coordination and Payout

A withdrawal coordinator claims pending requests and constructs a threshold-authorized payout transaction on Junocash. Each payout output carries `WithdrawalMemoV1`, which binds the output to the Base chain, bridge contract, withdrawal identifier, and batch context. This memo serves the same function as the deposit memo: it turns an otherwise opaque shielded output into one that can later be interpreted as belonging to a specific bridge withdrawal.

If a batch approaches expiry before broadcast is safe, operators can authorize an expiry extension through a separate typed-signature control path. Expiry constrains stale settlement records rather than forcing operators into unsafe payout timing.

6.2 Paid Marker and Finalization

After the Junocash payout confirms, the bridge records that state on Base through `markWithdrawPaidBatch`. This is deliberately distinct from finalization. A withdrawal may be known to have been paid on Junocash before Base has received or verified the proof required for final settlement. We separate these stages because they resolve different conditions: payout confirmation and conclusive Base-side settlement.

The finalizer then requests a proof and submits `finalizeWithdrawBatch`. The withdrawal proving path establishes that a Junocash output exists under the signed Orchard root, is recoverable under the bridge outgoing viewing surface, matches the intended recipient, and yields the expected net amount. The Base contract verifies the checkpoint quorum and proof, checks recipient hash equality against the stored raw recipient bytes, applies expiry or paid-marker rules, routes fees and relayer tip from escrow, burns the net amount, and marks the withdrawal finalized.

Table 5 summarizes the withdrawal stages.

Table 5: Withdrawal settlement stages.

Stage	Action	Main check
Request	User escrows `wJUNO` on Base	Recipient length, amount minimum
Coordination	Pending requests batched for payout	Threshold authorization
Paid marker	Base records Junocash-side payment	Sorted unique IDs, operator quorum
Finalization	`finalizeWithdrawBatch` executes	Proof, fee math, expiry, escrow burn

Expiry is a settlement gate rather than an alternate claim path. We separate escrow, payout, paid marking, and finalization because each stage resolves a different condition: value locking, Junocash-side payment, payment acknowledgment, and conclusive Base-side settlement.

7 Trust Model and Verification Boundary

Juno Intents is threshold-operated rather than trustless. This distinction defines the security model. Let n be the number of active operators and let t be the minimum number required to sign checkpoints and authorize threshold spends. If $O = \{1, 2, \ldots, n\}$ is the operator set and $S \subseteq O$ is the set participating in a given signing event, checkpoint acceptance requires $|S| \geq t$ with distinct active signers, as stated in Equation 2.

For a Junocash height h, the signed checkpoint tuple is C_h as defined in Equation 1. The core trust assumption is direct: if a malicious threshold quorum signs a false checkpoint, Base can accept a false view of Junocash state. We do not claim otherwise. We claim instead that this assumption is narrower, more inspectable, and more governable than the assumption of a single hot-wallet bridge operator.

7.1 Verification Boundary

The verification boundary has three components:

1. Operators attest to a checkpoint.
2. The proof system shows witness consistency relative to that checkpoint.
3. The Base contract decides whether the resulting public journal is an admissible bridge state transition.

This explains why checkpoint signatures and proofs coexist instead of replacing one another. The checkpoint determines which Junocash state commitment Base will act on. The proof establishes whether a claimed deposit or withdrawal is consistent with that committed state. The contract then determines whether that claim is admissible under bridge policy.

On the deposit side, the proof establishes that a note commitment is included under the signed Orchard root, that the note decrypts under the bridge incoming viewing key, that the memo parses to the intended Base recipient and domain, and that the amount and deposit identifier are derived deterministically:

$$\text{DepositItem} \Rightarrow \begin{cases} \text{note commitment} \in \text{signed Orchard root} \\ \text{note decrypts under bridge IVK} \\ \text{memo parses to intended Base domain} \\ \text{amount and } \texttt{depositId} \text{ derived deterministically} \end{cases} \tag{6}$$

On the withdrawal side:

$$\text{WithdrawItem} \Rightarrow \begin{cases} \text{output note} \in \text{signed Orchard root} \\ \text{memo binds withdrawal identifier and batch context} \\ \text{recipient and net amount match settlement package} \end{cases} \tag{7}$$

For a single accepted checkpoint C_h, the bridge relies on the following trust statement:

$$\exists\, S \subseteq O \;\text{ such that }\; |S| \geq t \;\text{ and every }\; i \in S \;\text{ signed }\; \text{Digest}(C_h). \tag{8}$$

The proof layer operates strictly relative to the signed Orchard root associated with that checkpoint.

7.2 Key Domain Separation

We use two distinct key domains, and the distinction is part of the security model. The first domain is Junocash custody: keys created through distributed key generation that authorize threshold spend activity from the bridge wallet. The second domain is Base operator signing: secp256k1 ECDSA keys used to sign EIP-712 checkpoint digests and related control messages, provisioned separately from threshold custody with support for KMS-backed signer workflows.

The security implication is straightforward. Compromise of a Base signer key does not directly spend Junocash custody shares. Compromise of one Junocash custody share does not directly authorize a Base checkpoint signature. The domains remain linked by operator identity and governance, but they are cryptographically separate.

Table 6: Key domain separation.

Domain	**Authority**	**Artifact**	**Compromise consequence**
Junocash custody	Threshold spend	Key-package material	Spend risk if threshold reached
Base signing	EIP-712 attestation	ECDSA signer lineage	False checkpoint risk if threshold reached

7.3 Checkpoint Frontier

The bridge maintains a checkpoint frontier on Base so that proofs attach to a single accepted sequence of signed state commitments rather than to an arbitrary collection of roots. The checkpoint digest is bound to the Base chain and bridge contract via EIP-712 domain separation. A checkpoint whose domain does not match the active deployment is rejected before any business logic is considered. Signatures must be sorted and unique, because threshold approval concerns distinct active operators, not repeated signatures from the same source.

Checkpoint progression is monotonic on Base. Once a checkpoint has been accepted, later submissions cannot move backward in height. A same-height submission with a different block hash or Orchard root is rejected as a conflict.

Table 7: Checkpoint frontier rules.

Rule	**Meaning**
Threshold signatures	At least t operator signatures required
Sorted unique signers	Duplicate or unordered signer sets rejected
Domain binding	Base chain ID and bridge contract are part of the digest
Monotonic height	Lower heights rejected after a checkpoint has been accepted
Conflict rejection	Same-height checkpoint must not change block hash or Orchard root

8 Proof Architecture

8.1 Role of SP1 in the Settlement Path

SP1 [7] is the proving system we use to generate a compact proof that a specific program was executed on specific hidden inputs and produced specific public outputs. In this bridge, SP1 is not used to prove all of Junocash consensus. It is used in a narrower way: to prove that a deposit or withdrawal statement is consistent with a signed Orchard root and with the bridge's parsing logic for notes, memos, and settlement records.

This narrower role is significant. Without a proving system, Base would have to trust relayers or operators to describe what happened inside Junocash shielded state. With SP1, Base can instead verify a cryptographic proof that a predefined guest program examined witness material under an accepted checkpoint root and emitted a specific public journal. SP1 therefore replaces unsupported testimony with a proof-carrying claim. It does not, however, make Base a Junocash light client or remove the need to trust the signed checkpoint root itself.

SP1 operates in a three-part chain. First, operators attest to a checkpoint root. Second, SP1 proves that a bridge-specific guest program saw witness data consistent with that root and produced a particular public journal. Third, the Base contract decides whether that journal is admissible under replay protection, fee rules, expiry rules, and settlement state. SP1 therefore strengthens the bridge by proving that a specific claim follows from the accepted root under the declared program, but it does not prove that the accepted root is the only honest Junocash state, and it does not replace contract policy.

8.2 Deposit and Withdrawal Guests

We use separate SP1 guest programs for the two settlement lanes because they prove different facts. In the deposit lane, the guest consumes witness material tied to the signed Orchard root, recomputes the root from the witness path, decrypts the note under the bridge incoming viewing key, parses `DepositMemoV1`, and emits a deposit journal carrying the (`depositId`, `recipient`, `amount`) values that Base uses for minting.

In the withdrawal lane, the guest works relative to the same signed Orchard root but with different semantics. It recovers the payout output under the bridge outgoing viewing surface, parses `WithdrawalMemoV1`, and emits a withdrawal journal carrying (`withdrawalId`, `recipientUAHash`, `netAmount`) for Base finalization.

8.3 Proving and Submission Path

The proving path is a transport stack around an SP1 program rather than a black-box oracle. The off-chain requestor builds the exact binary private input and the exact ABI journal expected by the contract, then submits both to the proving layer as a lane-specific request carrying the program identity, the journal, and the private input.

One detail is especially important. The proving layer does not invent the journal. The caller supplies the expected journal, the guest commits that journal as public values, and the resulting seal is meaningful only relative to that exact public journal. This allows Base to verify not merely that some proof passed, but that the submitted proof corresponds to the settlement record actually being claimed.

This architecture introduces an operational dependency as well as a cryptographic one. Proof requestors, prover adapters, and proof funders must remain available and sufficiently funded for the bridge to move. We acknowledge that this dependency affects liveness even though it does not change the meaning of the proof.

8.4 Bridge Verification Path on Base

Base verification is intentionally split between proof and contract logic. The guest proves witness validity relative to the signed Orchard root. The contract then decides whether that proved witness is admissible as a state transition. The on-chain proof verification surface is intentionally narrow, with the shape:

$$\texttt{verifyProof}(\texttt{programVKey}, \texttt{publicValues}, \texttt{proofBytes}). \tag{9}$$

The bridge binds each settlement lane to a distinct program identity through lane-specific image identifiers, and verifies both the public journal and the program identity before applying any contract state transition.

Table 8 summarizes the verification boundary.

Table 8: Verification boundary: what is proven, what is checked, and what is not proven.

Layer	Proven by guest	Checked by contract	Not proven
Checkpoint	N/A	Threshold signatures, domain, monotonic frontier	Honesty of root
Deposit	Inclusion, decryption, memo semantics, journal	Replay protection, minimums, fee routing	Consensus correctness
Withdrawal	Inclusion, output recovery, recipient binding, journal	Expiry, paid marker, fee math, escrow burn	Payout liveness

8.5 What the Proof Does Not Prove

We state this boundary explicitly. The proof does not prove that Base verifies Junocash consensus directly. It proves only that a guest saw a specific Orchard root and derived a specific journal from a valid witness under that root. The proof does not prove that a checkpoint root is honest; it proves only that the witness and the journal are consistent with the checkpoint root signed by

the operator quorum. The proof does not provide replay protection by itself; replay resistance comes from contract state. The proof does not enforce minimums, expiry policy, recipient policy, or fee routing; those remain explicit contract rules.

9 Distributed Custody and Auditability

9.1 Threshold Key Generation

The most important design decision in Juno Intents is that the bridge wallet is never introduced as a single secret key. The wallet is born through distributed key generation. In abstract threshold terms, the construction can be represented with a degree-$(t-1)$ polynomial:

$$f(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_{t-1} x^{t-1}. \tag{10}$$

Each operator receives one share $s_i = f(i)$ for $i \in O$. The group validating key corresponds to the constant coefficient:

$$ak = [a_0]\, G. \tag{11}$$

Any threshold subset S of size at least t can interpolate the constant term in the abstract Shamir model with Lagrange coefficients:

$$\lambda_i(S) = \prod_{\substack{j \in S \\ j \neq i}} \frac{0-j}{i-j}, \qquad a_0 = \sum_{i \in S} \lambda_i(S)\, s_i. \tag{12}$$

The operational signing path does not reconstruct a_0 during normal operation. Instead, threshold participants combine signing shares to produce one valid authorization. The abstract interpolation equations are included only to make the threshold property explicit.

9.2 Public Viewing Material

Threshold custody is paired with published viewing material. The bridge wallet publishes a Unified Full Viewing Key so that deposits, spends, and witnesses can be independently scanned. The deterministic derivations are:

$$nk = H_{\text{base}}(d_{nk} \parallel ak_{\text{bytes}}) \tag{13}$$

$$rivk = H_{\text{scalar}}(d_{rivk} \parallel ak_{\text{bytes}}) \tag{14}$$

where d_{nk} is the literal domain tag `WJUNO_NK_V1` and d_{rivk} is the literal domain tag `WJUNO_RIVK_V1`. The full viewing material is then constructed from $(ak, nk, rivk)$ and published as a UFVK.

The consequence is important. Auditability does not depend on trusting operator reports. Anyone with the published viewing material can observe the bridge wallet's note-level behavior. Because Junocash enforces mandatory shielding across the entire transaction pool [2], the UFVK audit surface covers all note activity relevant to the bridge wallet rather than only a shielded subset. At the same time, viewing does not imply spending; the threshold spend authority remains distributed among operators.

Figure 2 illustrates the distributed key generation ceremony.

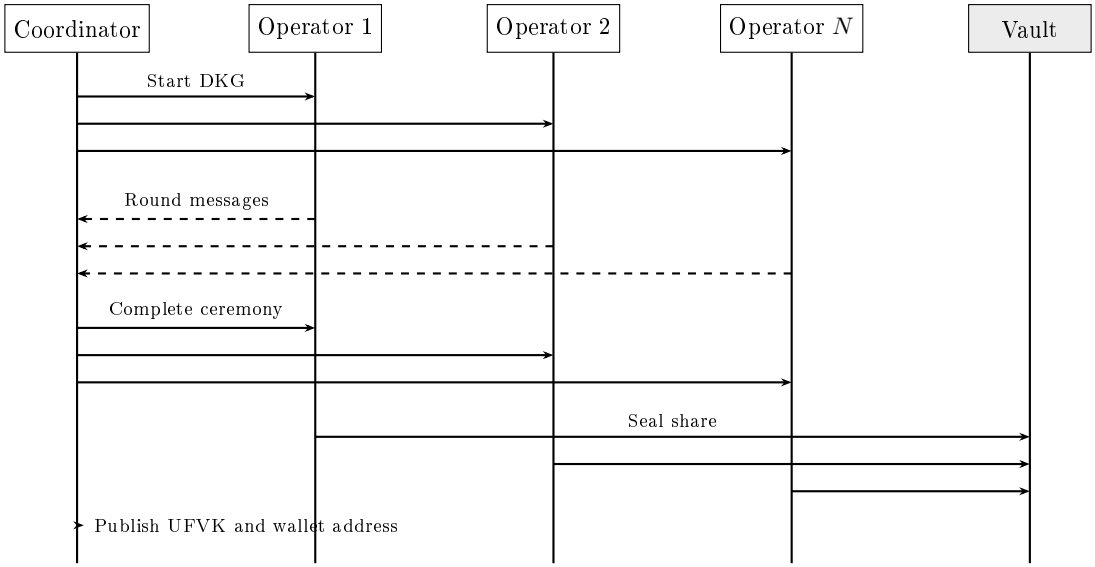

Figure 2: Distributed key generation ceremony. Operators exchange round messages through a coordinator, seal their shares to encrypted storage, and the coordinator publishes the viewing material and canonical wallet address.

10 Operator Economics and Governance

10.1 Fee Structure

We separate operator revenue from relayer work. The caller that submits mint or finalization transactions bears gas and execution risk and earns the relayer tip share (`relayerTipBps`). The remaining protocol fee share accrues to operators through the fee distribution path. This separation reflects the fact that relayers and operators perform different functions: relayers move transactions through Base, while operators sign checkpoints, participate in custody, and maintain the bridge's trusted surfaces.

Operator weights affect fee accounting, not checkpoint voting power. Checkpoint approval remains one signature per active operator with a separate threshold parameter. We state this directly because a weighted fee schedule can otherwise be mistaken for weighted quorum authority. The protocol readily supports equal-weight deployments, but that is a deployment choice rather than a protocol invariant.

10.2 Governance Surface

There is no staking or slashing layer in the current design. Operator discipline depends on governance removal, rotation, public monitoring, and reputation. We consider this an appropriate match for the current institutional context and describe it honestly rather than implying a punishment market that does not exist.

Governance on Base is explicit. The contract surface can pause bridge activity, manage operator membership and threshold state through the registry, update verifier dependencies, rotate program image identifiers, and adjust bridge parameters including minimums and fees. These powers are part of the trust model rather than incidental implementation details. A threshold-operated bridge requires a visible response path for incidents; without pause, rotation, and parameter control, the system would be forced either to continue operating through an anomaly or to rely on an undefined off-chain emergency process.

11 Ceremony Evidence and Recoverability

The distributed key-generation ceremony is more than a setup step. It is the point at which the bridge wallet identity, the threshold parameters, the operator participation set, and the resulting viewing surface are bound together. We treat the outputs of that ceremony as evidence-bearing artifacts rather than as transient operator notes. At minimum, the design expects durable records of which ceremony produced the active custody surface, what threshold and roster were in force, what bridge-wallet identity resulted, and what viewing material external observers should use for auditing.

This evidence function is especially important in a shielded system. In a transparent bridge, outsiders can reconstruct much of the operational story directly from chain data. In a shielded bridge, they cannot. Published viewing material, signed checkpoint packages, and stable ceremony lineage therefore play a larger explanatory role than they would in a transparent asset bridge.

Recoverability should be understood in the same terms. A threshold bridge that cannot restore enough durable custody material after operator turnover or infrastructure loss has failed in an economically relevant way even if no theft occurred. We therefore treat backup packages, restore validation, and key-refresh procedures as part of the system model rather than as auxiliary operator concerns.

We distinguish three classes of evidence:

- **Public evidence**: published viewing material, signed checkpoint packages, on-chain operator configuration, program-image identifiers, and observable settlement outcomes on Base.
- **Confidential operator evidence**: encrypted share packages, restore receipts, machine-specific signer provenance, and incident-response records.
- **Recovery evidence**: artifacts required to demonstrate that a key refresh, operator handoff, or controlled restore followed an intended procedure.

This distinction helps the reader understand what external audit can and cannot prove. External observers can verify that the bridge exposes a legible audit surface and that its public behavior is consistent with the published design. They cannot fully verify, from public data alone, that every confidential recovery or handoff procedure was executed correctly. We acknowledge that some trust in institutional process remains, even when the protocol exposes strong cryptographic and audit mechanisms.

12 Reliability and Failure Modes

Threshold systems trade unilateral compromise risk against coordination risk. The choice of operator set size and threshold determines not only security against compromise, but also liveness under outage. We analyze both dimensions for an illustrative 3-of-5 deployment.

If the per-operator compromise probability is p, the probability that at least three operators are compromised is:

$$P_{\text{compromise}}(p) = \sum_{k=3}^{5} \binom{5}{k} p^k (1-p)^{5-k}. \tag{15}$$

If the per-operator outage probability is q, the probability that threshold signing remains live is:

$$P_{\text{live}}(q) = \sum_{k=3}^{5} \binom{5}{k} (1-q)^k \, q^{5-k}. \tag{16}$$

Selected values are shown in Table 9, and Figures 3 and 4 illustrate the full curves. These curves are not claims about exact production behavior. Real systems exhibit correlated outages,

correlated operator risk, and non-independent failure domains. Nevertheless, the curves make the central operating trade-off explicit: a bridge that resists unilateral compromise also accepts that liveness depends on enough independent operators staying available.

Table 9: Compromise and liveness probabilities for a 3-of-5 operator set.

Per-operator probability	$P_{\text{compromise}}$	P_{live}
5%	0.1158%	99.8842%
10%	0.8560%	99.1440%
20%	5.7920%	94.2080%
30%	16.3080%	83.6920%

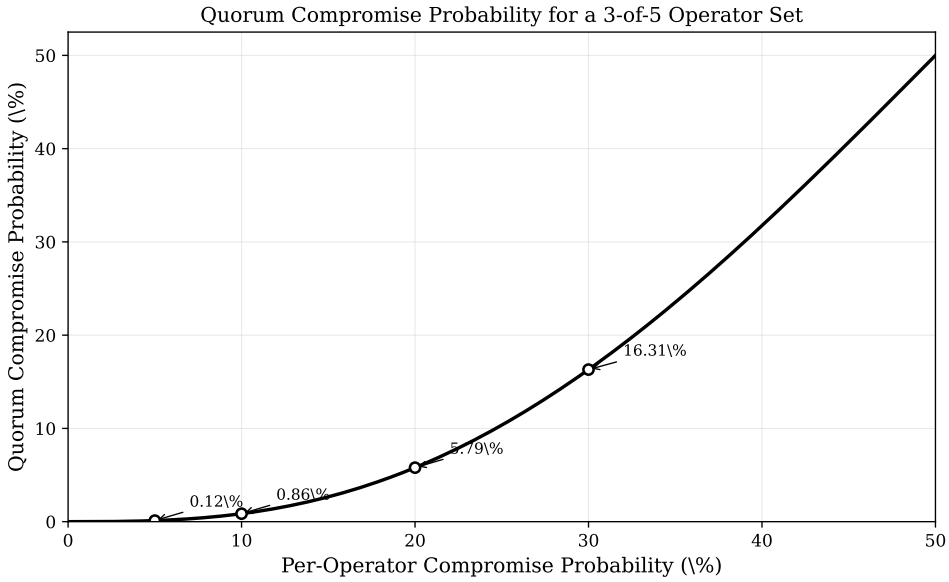

Figure 3: Quorum compromise probability for a 3-of-5 operator set. The quorum condition compounds the benefit of reducing per-operator compromise probability.

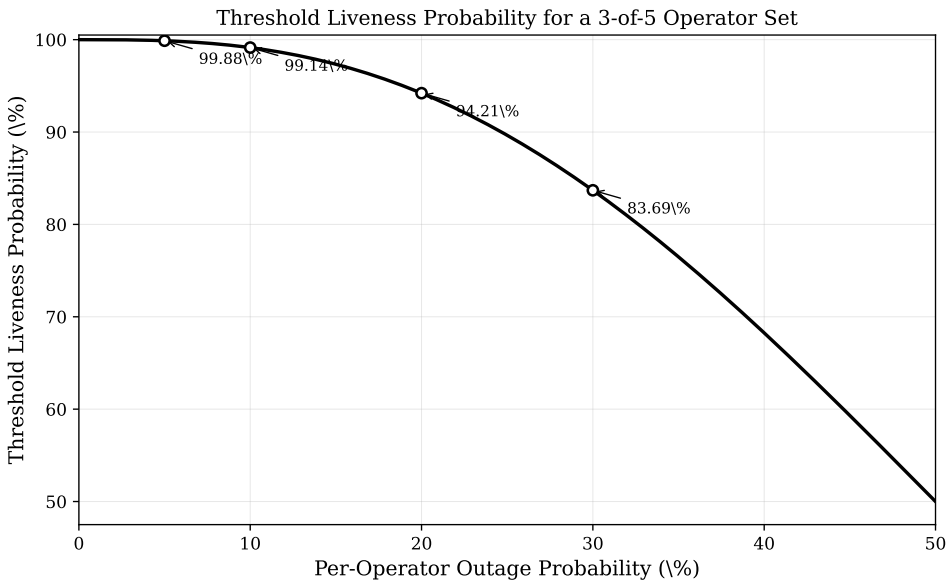

Figure 4: Threshold liveness probability for a 3-of-5 operator set. The same threshold that reduces unilateral custody risk creates a liveness floor: correlated outages across three operators halt signing.

Three observations guide operational planning. First, lowering per-operator compromise probability matters non-linearly because the quorum condition compounds the benefit. Second, the same threshold that reduces unilateral custody also creates a liveness floor: correlated outages across three operators halt signing. Third, the system must therefore optimize both for confidentiality and for durable recoverability of enough shares.

12.1 Failure Modes

The main failure classes are:

Malicious checkpoint quorum. A threshold of operators signs a false checkpoint, allowing Base to accept a false Junocash state commitment. This is the most serious failure mode and the core reason we describe the system as threshold-operated rather than trustless. The mitigations are public monitoring, governance response, rapid pausing, and key rotation.

Custody-share loss. Enough operators lose recoverable share material that Junocash payouts halt even without theft. This is an availability failure rather than a safety failure, but it remains economically important.

Partial domain compromise. A Base signing key or Junocash custody share is compromised, reducing the safety margin even if full threshold compromise has not occurred. Either event should trigger investigation and, where appropriate, rotation.

Proof-path outage. Proving, proof submission, or proof funding failures stall minting and finalization. This is an operational dependency rather than a custody failure, but a production system must treat it as seriously as a cryptographic concern.

Deep reorganization. A signed checkpoint later diverges from chain reality through an unexpected reorganization, requiring pause and governance intervention. Junocash operates with 60-second block times and a 100-block difficulty adjustment window [2], so checkpoint confirmation policy should account for these parameters when setting the minimum confirmation depth before signing. We assume conservative confirmation policy and active monitoring of reorg anomalies.

12.2 Rotation Procedure

When quorum honesty or share safety is in doubt, the bridge follows this procedure:

1. Freeze new withdrawal execution.
2. Run a new threshold-custody refresh for the relevant operator change or suspected compromise.
3. Publish refreshed canonical viewing material and the replacement bridge-wallet identity.
4. Update the on-chain operator and keyset configuration before resuming normal flow.

13 Comparative Positioning

Juno Intents sits between two familiar bridge extremes. At one extreme is the single-signer or tightly centralized bridge, which is operationally simple but concentrates custody and attestation risk into one party. Its simplicity comes at the cost of requiring users to trust a single control surface not to fail, misbehave, or be coerced. At the other extreme is a fully consensus-verifying light-client bridge, which minimizes external trust but is substantially more demanding in proof and verification complexity, especially when the source chain uses shielded state with nontrivial witness semantics.

We occupy an intermediate position. We do not ask Base to verify Junocash consensus directly. Instead, we narrow trust into a threshold checkpoint layer and use proofs to verify deposit and withdrawal semantics relative to that layer. The central claim is threshold-operated rather than trustless bridging, with explicit and inspectable trust for shielded-asset settlement.

This positioning is especially relevant for shielded assets. Once deposits and withdrawals are not natively legible to the destination chain, the problem is less about removing all trust at once and more about choosing where trust sits, how it is monitored, and how it can be rotated or paused when assumptions fail.

14 Conclusion

We have presented Juno Intents, a bridge architecture for shielded Junocash liquidity that is candid about what is and is not proven. Threshold custody reduces single-party spend risk on the Junocash side. Signed checkpoints create an explicit Base-side acceptance boundary. SP1 proofs bind concrete deposit and withdrawal events to that boundary. Published viewing material makes bridge activity more legible to external observers. Governance and rotation powers make incident response part of the system design rather than an emergency measure.

The design remains threshold-operated rather than trustless. It narrows and clarifies the trust boundary relative to a conventional single-signer shielded bridge. In that more limited but practical sense, Juno Intents provides a more disciplined cryptographic and operational foundation for shielded-asset bridging.

A Canonical Encodings

This appendix captures the protocol encodings that matter for routing, domain separation, replay resistance, and settlement interpretation.

A.1 `DepositMemoV1`

`DepositMemoV1` is a fixed 512-byte memo format. It is big-endian, CRC32-protected, zero-padded, and domain-separated by both Base chain ID and bridge address.

Table 10: `DepositMemoV1` encoding.

Offset	Length	Field	Notes
`0x00`	8	`magic`	`WJUNO\x01\x00\x00`
`0x08`	4	`baseChainId`	Big-endian `u32`
`0x0c`	20	`bridgeAddr`	Domain binding
`0x20`	20	`baseRecipient`	Recipient on Base
`0x34`	8	`nonce`	Validated, not surfaced in journal
`0x3c`	4	`flags`	Validated, not surfaced in journal
`0x40`	4	`crc32`	IEEE CRC over bytes `0x00..0x3f`
`0x44`	...	padding	Must be zero

A.2 `WithdrawalMemoV1`

`WithdrawalMemoV1` is also a fixed 512-byte memo format with the same checksum and padding discipline.

Table 11: `WithdrawalMemoV1` encoding.

Offset	Length	Field	Notes
`0x00`	8	`magic`	`WJUNOWD\x01`
`0x08`	4	`baseChainId`	Big-endian `u32`
`0x0c`	20	`bridgeAddr`	Domain binding
`0x20`	32	`withdrawalId`	Must match requested withdrawal
`0x40`	32	`batchId`	Payout builder use only
`0x60`	4	`flags`	Validated, not surfaced in journal
`0x64`	4	`crc32`	IEEE CRC over bytes `0x00..0x63`
`0x68`	...	padding	Must be zero

A.3 Deposit and Withdrawal Identifiers

The canonical deposit identifier is given in Equation 4. The canonical withdrawal identifier is given in Equation 5.

The canonical withdrawal batch identifier is:

$$\texttt{withdrawalIdsHash} = \text{Keccak256}\big(\text{concat}(\text{sorted withdrawal IDs})\big) \tag{17}$$

$$\texttt{batchId} = \text{Keccak256}\big(\texttt{"WJUNO_WITHDRAW_BATCH_V1"} \parallel \texttt{withdrawalIdsHash}\big) \tag{18}$$

A.4 Checkpoint and Control-Message Families

Checkpoint tuple fields are defined in Section 3. Base-side operational control-message families are:

Table 12: Control-message families.

Family	Payload	Purpose
`ExtendWithdrawExpiry`	IDs hash, new expiry, chain, bridge	Extend expiry before payout
`MarkWithdrawPaid`	IDs hash, chain, bridge	Record Junocash-side payment

For both families:

$$\texttt{withdrawalIdsHash} = \text{Keccak256}\big(\text{abi.encodePacked(sorted unique withdrawal IDs)}\big). \quad (19)$$

A.5 Guest Private Inputs

Production guests consume raw binary private input. The deposit guest header layout is:

```
finalOrchardRoot[32] || baseChainIdLE[4] || bridgeContract[20]
  || owalletIVK[64] || nLE[4] || n * depositItem
```

The withdraw guest header layout is:

```
finalOrchardRoot[32] || baseChainIdLE[4] || bridgeContract[20]
  || owalletOVK[32] || nLE[4] || n * withdrawItem
```

Table 13: Witness item layouts.

Item	Layout
Deposit	`leafIndexLE[4] || authPath[32*32] || nf[32] || rk[32] || cmx[32] || epk[32] || encCiphertext[580] || outCiphertext[80] || cvNet[32]`
Withdraw	`withdrawalId[32] || recipientRawAddress[43] || leafIndexLE[4] || authPath[32*32] || nf[32] || rk[32] || cmx[32] || epk[32] || encCiphertext[580] || outCiphertext[80] || cvNet[32]`

A.6 Public Journals

The deposit public journal is:

`DepositJournal(finalOrchardRoot, baseChainId, bridgeContract, items[])`

where each item is (`depositId`, `recipient`, `amount`).

The withdraw public journal is:

`WithdrawJournal(finalOrchardRoot, baseChainId, bridgeContract, items[])`

where each item is (`withdrawalId`, `recipientUAHash`, `netAmount`). The value `recipientUAHash` is the Keccak-256 hash of the raw 43-byte Orchard recipient address, not of a display string.

A.7 Hard Caps

Table 14: Protocol hard caps.

Bound	Value	Scope
Guest deposit items	100	Guest input encoding and guest code
Guest withdraw items	100	Guest input encoding and guest code
Bridge mint/finalize batch	200	`mintBatch` and `finalizeWithdrawBatch`
Control-message batch	200	`extendWithdrawExpiryBatch` and `markWithdrawPaidBatch`

B Implementation Anchor Map

This appendix identifies source locations in the companion implementation that correspond to the paper's main claims.

Table 15: Implementation anchors.

Paper topic	Primary anchor
EIP-712 checkpoint and control-message digests	`internal/checkpoint/checkpoint.go`, `mark_paid.go`
Deposit and withdrawal memo formats	`internal/memo/memo.go`
Witness extraction	`internal/witnessextract/extractor.go`
Guest private-input encoding	`internal/proverinput/private_input.go`
Deposit settlement and `mintBatch`	`internal/depositrelayer/relayer.go`, `contracts/src/Bridge.sol`
Withdrawal coordination and paid-marker flow	`internal/withdrawcoordinator/coordinator.go`, `paid_marker.go`, `planner_txbuild.go`
Withdrawal finalization	`internal/withdrawfinalizer/finalizer.go`, `contracts/src/Bridge.sol`
Operator registry and governance	`contracts/src/OperatorRegistry.sol`, `contracts/src/Bridge.sol`
Integration tests	`contracts/test/integration/BridgeFlow.t.sol`, `cmd/bridge-e2e/main.go`

References

[1] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008. Available: https://bitcoin.org/bitcoin.pdf

[2] The Juno Cash Project, "Juno Cash: Peer-to-peer electronic cash with mandatory privacy and egalitarian mining," 2025. Available: https://juno.cash/junocash.pdf

[3] Electric Coin Company, "Zcash protocol specification," 2024. Available: https://zips.z.cash/protocol/protocol.pdf

[4] J. Grigg, S. Bowe, D. Hopwood, and J. Jelinek, "ZIP 224: Orchard shielded protocol," 2021. Available: https://zips.z.cash/zip-0224

[5] D. Hopwood and J. Grigg, "ZIP 316: Unified addresses and unified viewing keys," 2021. Available: https://zips.z.cash/zip-0316

[6] R. Floersch and B. Whinfrey, "EIP-712: Typed structured data hashing and signing," 2017. Available: https://eips.ethereum.org/EIPS/eip-712

[7] Succinct Labs, "SP1 documentation," 2024. Available: https://docs.succinct.xyz/docs/sp1/introduction

[8] A. Zamyatin, D. Harz, J. Lind, P. Panber, A. Gervais, and W. Knottenbelt, "XCLAIM: Trustless, interoperable, cryptocurrency-backed assets," in *IEEE Symposium on Security and Privacy*, 2019.

[9] A. Shamir, "How to share a secret," *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, 1979.

[10] S. Bowe, J. Grigg, and D. Hopwood, "Recursive proof composition without a trusted setup," *Cryptology ePrint Archive*, Report 2019/1021, 2019.